

Visible
Value chain
Invisible

Uncharted

Outcome-Oriented Programming for Agentic Execution

A Wardley map of the hypothesis. **Hypothesis, not forecast.** The horizontal axis is evolution, not time.

Outcome-Oriented Programming for Agentic Execution

new execution substrate

Outcome-oriented agentic execution
(interpret, plan, lower, bind, verify, gate)

Development:
Developer + LLM

User / organization
(person, workload, machine, device, robot, vehicle)

value-chain anchors: horizontal position is not a maturity claim

Desired business outcome

Application-centric composition /
pre-authored workflows

inertia: organization, security,
procurement and ownership are
organized around applications

Existing enterprise systems,
processes and data

standardising

Intent + authority
context C_0

Semantic intent,
ontology binding

Frontier models
(differentiated)

Agent / orchestration
frameworks

IER: Intent Execution
Representation

Semantic Manifests,
tool contracts

Verifier-oriented
Intent Runtime

Deterministic tools
(software and physical)

Mainstream / open / local
model access

Runtime governance for AI agents:
Execution Gates, Governance
Intelligence, dynamic risk decisions

Application containment,
behavioral verification

Formal verification
(Lean, Tamarin, ProVerif;
PIC proof stack)

PIC: authority continuity,
lineage, explicit composition

Identity and authorization
(OAuth, tokens: possession)

Transport, queues, messaging

carries execution;
not authorization semantics
Cloud compute,
storage, network

PIC-X exchange
(existing authority $\rightarrow C_0$)

Open

public specs, proofs

data centres, energy;
sovereign capacity fragments supply

+constraint

AI inference hardware
(GPUs, data centres)

Genesis Custom built Product (+ rental) Commodity (+ utility) Evolution

From pre-authored applications to runtime-composed outcomes

probabilistic intent and planning
 $\rightarrow$ explicit semantics / IER
 $\rightarrow$ authorized, verifiable, governed exe-
cution

The application stops being the manda-
tory unit of composition; stable work
crystallizes into reusable tools.

Legend

- dependency (current)
- dependency (hypothesised, OOP)
- - - evolution of the same component
- █ inertia
- [- - -] emerging higher-order system
- gameplay: open, +constraint

Genesis begets evolution begets genesis. Industrialised compute and increas-
ingly productised model access, together with existing deterministic enterprise
systems, enable the new genesis on the left. The right side is not the future:
the left emerges because the right has become industrialised.